In [1]:
```python
# CARGA DE DATOS #

import pandas as pd
# test comment
# Ruta al archivo de Excel
archivo_excel = './IIYNT-REQ-BD-003.xlsx' #../

# Número de la fila desde la cual querés empezar (0-indexado)
fila_inicio = 3

# Leer el archivo Excel
init_df = pd.read_excel(archivo_excel)

# Mostrar las primeras filas del DataFrame para confirmar que se leyó bien
init_df.head()

# TRANSFORMACIÓN DE DATOS #

desired_columns = [
    'Temperatura Ambiental (°C)',
    'Humedad',
    'Temperatura de la muestra (°C)',
    'pH',
    'CE\n(µS/cm)',
    'STD\n(mg/L)',
    'STS\n(mL sed/L)',
    'OD\n(mg/L)',
    'Nivel (cm)',
    'Turbidez (NTU)',
    'Dureza\n(mg CaCO3/L)',
    'Cloruros\n(mg Cl-/L)'
]
df = init_df.iloc[2:123][desired_columns]
df
```

Out[1]:

	Temperatura Ambiental (°C)	Humedad	Temperatura de la muestra (°C)	pH	CE\n(µS/cm)	STD\n(mg/L)	STS\n(mL sed/L)	OD\n(mg/L)
2	16	0.929	18	8.2	1240	610	NaN	5.37
3	17	0.47	19	8.3	1630	810	1.8	4.3
4	11.9	0.47	13	8.1	1000	490	18	5.3
5	11.9	0.47	13	8.2	1000	490	18	4.67
6	11.9	0.47	13	8.3	1350	670	0.1	7.01
...	...	...	...	...	...	...	...	...
118	12.8	0.57	15.3	8	1110	550	18	4.18
119	12.8	0.57	15.3	8.1	1080	550	18	4.23
120	12.8	0.57	15.4	8.4	1050	520	44	1.96
121	12.8	0.57	15.4	8.4	1070	530	44	1.67
122	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

121 rows × 12 columns

In [2]:
```python
#ver los que tienen valores nulos o no asignados

#df.isnull().sum()
df.isna().sum()
```

```python
#Eliminar las filas que tienen valores nulos o no asignados
#df.dropna(inplace=True) #aca se modifica el Dataframe original con inplace=true

#Si asigno a nueva variable no hace falta inplace=true pero no modifica el Dataframe
#df_sin_nulos=df.dropna()

#Como opcion puedo llenar los valores nulos con datos adecuados:
#fillna(method='bfill') (llenar con el valor siguiente de la columna)
#fillna(method='ffill') (llenar con el valor anterior de la columna)
#fillna(0) (llenar con ceros)

mediaNivel=df['Nivel (cm)'].mean()
df['Nivel (cm)']=df['Nivel (cm)'].fillna(mediaNivel)

mediaSTS=df['STS\n(mL sed/L)'].mean()
df['STS\n(mL sed/L)']=df['STS\n(mL sed/L)'].fillna(mediaSTS)

mediaTurbidez=df['Turbidez (NTU)'].mean()
df['Turbidez (NTU)']=df['Turbidez (NTU)'].fillna(mediaTurbidez)

mediaDureza=df['Dureza\n(mg CaCO3/L)'].mean()
df['Dureza\n(mg CaCO3/L)']=df['Dureza\n(mg CaCO3/L)'].fillna(mediaDureza)

mediaCloruros=df['Cloruros\n(mg Cl-/L)'].mean()
df['Cloruros\n(mg Cl-/L)']=df['Cloruros\n(mg Cl-/L)'].fillna(mediaCloruros)

df_sin_nulos=df.copy()
df_sin_nulos.isna().sum()

#Eliminar las filas que tienen valores nulos o no asignados
df_sin_nulos.dropna(inplace=True) #aca se modifica el Dataframe original con inplace=
df_sin_nulos.isna().sum()
```

Out[2]:

```
Temperatura Ambiental (°C)        0
Humedad                           0
Temperatura de la muestra (°C)    0
pH                                0
CE\n(µS/cm)                       0
STD\n(mg/L)                       0
STS\n(mL sed/L)                   0
OD\n(mg/L)                        0
Nivel (cm)                        0
Turbidez (NTU)                    0
Dureza\n(mg CaCO3/L)              0
Cloruros\n(mg Cl-/L)              0
dtype: int64
```

In [3]:

```python
def min_max_info(dataframe: pd.DataFrame) -> pd.DataFrame:
    name = list()
    minimums = list()
    maximums = list()
    for i, key in enumerate(dataframe.columns):
        name.append(key)
        minimums.append(dataframe[key].min())
        maximums.append(dataframe[key].max())
    return pd.DataFrame(
        {
            'name': name,
            'mins': minimums,
            'maxs': maximums
        }
    )
min_max_info(df_sin_nulos)
```

Out[3]:

	name	mins	maxs
0	Temperatura Ambiental (°C)	10.40	26.000
1	Humedad	0.36	0.929
2	Temperatura de la muestra (°C)	12.80	28.100
3	pH	7.20	8.700
4	CE\n(µS/cm)	200.00	1690.000
5	STD\n(mg/L)	140.00	840.000
6	STS\n(mL sed/L)	0.10	650.000
7	OD\n(mg/L)	0.00	9.120
8	Nivel (cm)	20.00	70.000
9	Turbidez (NTU)	2.68	1000.000
10	Dureza\n(mg CaCO3/L)	86.00	316.000
11	Cloruros\n(mg Cl-/L)	15.00	156.000

In [4]:

```
def normalize(dataframe: pd.DataFrame, min: float, max: float) -> pd.DataFrame:
    return (dataframe - min) / (max - min)

def denormalize(dataframe: pd.DataFrame, min: float, max: float) -> pd.DataFrame:
    return dataframe * (max - min) + min


def normalizer(dataframe: pd.DataFrame) -> pd.DataFrame:
    df = dataframe.copy()
    df['Temperatura Ambiental (°C)']                = normalize(df['Temperatura Ambie
    df['Humedad'] = df['Humedad']
    df['Temperatura de la muestra (°C)']            = normalize(df['Temperatura de la
    df['pH']                                        = normalize(df['pH'],
    df['CE\n(µS/cm)']                               = normalize(df['CE\n(µS/cm)'],
    df['STD\n(mg/L)']                               = normalize(df['STD\n(mg/L)'],
    df['STS\n(mL sed/L)']                           = normalize(df['STS\n(mL sed/L)']
    df['OD\n(mg/L)']                                = normalize(df['OD\n(mg/L)'],
    df['Nivel (cm)']                                = normalize(df['Nivel (cm)'],
    df['Turbidez (NTU)']                            = normalize(df['Turbidez (NTU)'],
    df['Dureza\n(mg CaCO3/L)']                      = normalize(df['Dureza\n(mg CaCO3
    df['Cloruros\n(mg Cl-/L)']                      = normalize(df['Cloruros\n(mg Cl-
    return df

def denormalizer(dataframe: pd.DataFrame) -> pd.DataFrame:
    df = dataframe.copy()
    df['Temperatura Ambiental (°C)']                = denormalize(df['Temperatura Amb
    df['Humedad'] = df['Humedad']
    df['Temperatura de la muestra (°C)']            = denormalize(df['Temperatura de
    df['pH']                                        = denormalize(df['pH'],
    df['CE\n(µS/cm)']                               = denormalize(df['CE\n(µS/cm)'],
    df['STD\n(mg/L)']                               = denormalize(df['STD\n(mg/L)'],
    df['STS\n(mL sed/L)']                           = denormalize(df['STS\n(mL sed/L)
    df['OD\n(mg/L)']                                = denormalize(df['OD\n(mg/L)'],
    df['Nivel (cm)']                                = denormalize(df['Nivel (cm)'],
    df['Turbidez (NTU)']                            = denormalize(df['Turbidez (NTU)'
    df['Dureza\n(mg CaCO3/L)']                      = denormalize(df['Dureza\n(mg CaC
    df['Cloruros\n(mg Cl-/L)']                      = denormalize(df['Cloruros\n(mg C
    return df
```

In [5]:

```
from keras.layers import Input, Dense # tipos de capas de la red neuronal:
                                      # Dense = Conectado todo con todo
                                      # Input = Solo entrada
from keras.activations import sigmoid # Función de activación a la salida de la neuro
from keras.losses import mse # Tipo de ajuste de error (mean square error)
```

```python
from keras.models import Model # Clase que devuelve la instancia del modelo
                               # el modelo es de la forma entrada -> salida
import numpy as np
import os as os

from tensorflow.keras.callbacks import TensorBoard # Interfaz Web de Tensorflow para
                                                   # grafico de entrenamiento
import shutil # Utilidad para manejo de consola
try:
    shutil.rmtree('./.logs') # Si existe, borro la carpeta .logs que es necesaria
                             # para la ingesta de tensorboard
except FileNotFoundError:
    pass
os.mkdir('./.logs') # creo una carpeta vacia
                    # para entrenar, ejecutar en consola y dentro del entorno virtual
                    # lo siguiente: tensorboard --logdir=./.logs
vector_size = len(df_sin_nulos.columns) # defino tamaño de vector de datos (en este c
dataset = np.array( # Lo hago matriz, papu
    normalizer(df_sin_nulos).astype(np.float16) # Normalizo el dataset inicial sin nu
)
encoded_size = 3 # Me invento un tamaño para el vector clasificador

# definimos modelo: (notación algebraica simulada: x = f(y))
# preguntar a chatGPT el tema de los shapes de keras porque es un lio
input_layer = Input(shape=(vector_size, )) # buscar en chatgpt maneras de construir u

x = Dense(4, activation=sigmoid)(input_layer) # se colocan unicamente 4 neuronas. Est
# nota: Instancio la clase y la llamo, esto es un diseño "raro" de keras para que la
# de la red se "parezca" a una notacion matematica

encoded = Dense(encoded_size, activation=sigmoid)(x) #seria como la matriz de pesos p
x = Dense(4, activation=sigmoid)(encoded)
output = Dense(vector_size, activation=sigmoid)(x) # metodo call

autoencoder = Model(input_layer, output) # creo un modelo a partir de lo definido arr
autoencoder.compile(optimizer='adam', loss='mse') # se compila el modelo
autoencoder.summary() # se imprime un resumen
```

```
2023-10-25 16:16:46.092827: I tensorflow/core/util/port.cc:110] oneDNN custom operatio
ns are on. You may see slightly different numerical results due to floating-point roun
d-off errors from different computation orders. To turn them off, set the environment
variable `TF_ENABLE_ONEDNN_OPTS=0`.
2023-10-25 16:16:46.094893: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find cu
da drivers on your machine, GPU will not be used.
2023-10-25 16:16:46.131939: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find cu
da drivers on your machine, GPU will not be used.
2023-10-25 16:16:46.132895: I tensorflow/core/platform/cpu_feature_guard.cc:182] This
TensorFlow binary is optimized to use available CPU instructions in performance-critic
al operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operation
s, rebuild TensorFlow with the appropriate compiler flags.
2023-10-25 16:16:46.856236: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF
-TRT Warning: Could not find TensorRT
```

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 12)]              0

 dense (Dense)               (None, 4)                 52

 dense_1 (Dense)             (None, 3)                 15

 dense_2 (Dense)             (None, 4)                 16

 dense_3 (Dense)             (None, 12)                60

=================================================================
Total params: 143 (572.00 Byte)
Trainable params: 143 (572.00 Byte)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

In [6]:

```
tensorboard = TensorBoard(log_dir='./.logs') #creo instancia para guardar
epochs = 12000 # cantidad de veces a entranar o pasos para realizar el descenso del g
autoencoder.fit(
    x=dataset,
    y=dataset,
    epochs=epochs,
    batch_size=len(dataset), #tamaño del bloque de entrenamiento
    shuffle=True,
    verbose=0, # que no envie salidas
    callbacks=[tensorboard]

)
```

Out[6]:

```
<keras.src.callbacks.History at 0x7fcf10093910>
```

In [7]:

```
generator = Model(encoded, output) #va del vector de clasificacion hasta la salida
generator.summary()
```

```
Model: "model_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_2 (InputLayer)        [(None, 3)]               0

 dense_2 (Dense)             (None, 4)                 16

 dense_3 (Dense)             (None, 12)                60

=================================================================
Total params: 76 (304.00 Byte)
Trainable params: 76 (304.00 Byte)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

In [8]:

```
encoder = Model(input_layer, encoded) #va desde el input a los datos codificados o cl
encoder.summary()
```

```
Model: "model_2"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 12)]              0

 dense (Dense)               (None, 4)                 52

 dense_1 (Dense)             (None, 3)                 15

=================================================================
Total params: 67 (268.00 Byte)
Trainable params: 67 (268.00 Byte)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

In [9]:

```
amount = 10000 #cantidad de predicciones
generated = np.zeros(shape=(amount, vector_size))
params = np.random.rand(amount, encoded_size)
generated = generator.predict(params)
gen_df = pd.DataFrame(generated)
gen_df.columns = df_sin_nulos.columns
gen_df = denormalizer(gen_df)
gen_df.to_csv('./dataset_simulado_3_neuronas.csv')
gen_df
```

```
313/313 [==============================] - 0s 576us/step
```

Out[9]:

	Temperatura Ambiental (°C)	Humedad	Temperatura de la muestra (°C)	pH	CE\n(µS/cm)	STD\n(mg/L)	STS\n(mL sed/L)	OD\r
0	23.179438	0.336361	21.929760	7.928641	1781.013184	888.534241	87.845795	0.
1	22.286655	0.584384	19.498196	7.860925	1619.460815	803.914062	165.425903	0.
2	12.679802	0.631287	15.598328	8.148006	481.376709	252.910675	23.807198	7.
3	13.377996	0.215585	19.077847	7.890761	557.131470	288.339569	20.733137	0.
4	22.163536	0.418348	20.946131	7.961164	1661.511841	825.519287	77.673836	0.
...	...	...	...	...	...	...	...	
9995	15.229922	0.618457	16.299389	8.374810	1739.624390	869.066162	55.885712	5.
9996	25.402790	0.574937	20.391903	7.538157	1766.763062	881.922546	485.305084	0.
9997	22.627886	0.540847	20.120159	8.165709	1878.313965	939.024780	93.821968	0.
9998	14.617381	0.217124	20.129192	7.874763	315.110840	182.524231	10.432275	0.
9999	13.304130	0.268438	18.272306	7.969954	885.652100	441.356873	31.540190	1.

10000 rows × 12 columns

In [10]:

```
min_max_info(df_sin_nulos)
```

Out[10]:

	name	mins	maxs
0	Temperatura Ambiental (°C)	10.40	26.000
1	Humedad	0.36	0.929
2	Temperatura de la muestra (°C)	12.80	28.100
3	pH	7.20	8.700
4	CE\n(µS/cm)	200.00	1690.000
5	STD\n(mg/L)	140.00	840.000
6	STS\n(mL sed/L)	0.10	650.000
7	OD\n(mg/L)	0.00	9.120
8	Nivel (cm)	20.00	70.000
9	Turbidez (NTU)	2.68	1000.000
10	Dureza\n(mg CaCO3/L)	86.00	316.000
11	Cloruros\n(mg Cl-/L)	15.00	156.000

In [11]:

```
min_max_info(gen_df)
```

Out[11]:

	name	mins	maxs
0	Temperatura Ambiental (°C)	7.790172	26.369593
1	Humedad	0.172622	0.876711
2	Temperatura de la muestra (°C)	14.033363	22.718834
3	pH	7.196640	8.488577
4	CE\n(µS/cm)	204.453003	1905.546387
5	STD\n(mg/L)	138.997650	952.941223
6	STS\n(mL sed/L)	4.771408	800.898315
7	OD\n(mg/L)	0.008730	9.647984
8	Nivel (cm)	15.049879	76.993439
9	Turbidez (NTU)	5.503571	993.987244
10	Dureza\n(mg CaCO3/L)	92.663818	360.318390
11	Cloruros\n(mg Cl-/L)	3.402330	189.613998

In [12]:

```
colors = encoder.predict(dataset)
colored_dataset = df_sin_nulos.copy() #copio el dataset original y lo coloreo con el



def color_row(row, color_vector):
    color_rgb = tuple((color_vector*255).astype(np.int16))
    color_hex = '#{:02X}{:02X}{:02X}'.format(color_rgb[0], color_rgb[1], color_rgb[2]
    return [f'background-color: {color_hex};' for _ in row.index]

# Create an iterator for the colors
color_iterator = iter(colors)

# Apply colors to each row
styled_df = colored_dataset.style.apply(lambda row: color_row(row, next(color_iterato
styled_df.to_excel("colored_dataset_3_neuronas.xlsx", engine='openpyxl')
```

```
4/4 [==============================] - 0s 1ms/step
```